

OPTIMUS

Autonomous Software Engineer

Complete Product, Architecture & Development Specification

Personal Portfolio Project

Core promise

From a real repository issue to a tested, reviewable and verified software change.

Document Map

Section	What it contains
1. Product Definition	Executive summary, problem, solution, differentiator, objectives
2. Product Workflow	End-to-end lifecycle and state machine
3. UI/UX	Final screen map and responsibilities
4. Technical Architecture	Frontend, backend, orchestrator, worker, context, AI
5. Repository Intelligence	Indexing, symbol graph, relevance retrieval, context assembly
6. AI System	Provider abstraction, prompt contracts, token/cost policy
7. Execution & Security	Sandbox, commands, limits, cleanup, threat model
8. Database & APIs	Prisma schema concept, entities, REST contracts
9. File Structure	Final JavaScript/JSX project structure and phases
10. Engineering Plan	Roadmap, tests, benchmark, deployment and acceptance criteria
11. Future Scope	Extensions after the MVP

1. Product Definition

1.1 Executive Summary

OPTIMUS is an AI-assisted autonomous software engineering platform for working with real software repositories. A developer provides a GitHub repository or project archive and describes a bug, feature, refactor or improvement request. **OPTIMUS** builds repository context, identifies relevant files and dependencies, creates a reviewable implementation plan, waits for human approval, applies changes in an isolated execution environment, runs tests and other checks, diagnoses failures, performs corrective iterations when necessary, and produces a verified patch with evidence.

The product is deliberately not positioned as another code generator. Its product value comes from the complete engineering loop: understand the repository, plan the change, implement it, test it, react to failures, verify the result and hand the change back to the developer in a reviewable form.

Portfolio positioning

INTELLIVORA demonstrates an AI application for interviews. OMNIX demonstrates a multi-agent AI platform.

OPTIMUS demonstrates AI-assisted software engineering and autonomous task execution.

1.2 Exact Problem Statement

Modern software projects contain many files, modules, dependencies, APIs, configuration files and tests. Fixing a real issue therefore requires more than producing a code snippet. A developer must first understand the repository, locate the relevant code, reason about dependencies, plan a safe change, edit files, run tests, inspect failures, iterate and verify the outcome. Generic coding assistants can accelerate individual steps, but a developer still has to coordinate the end-to-end process.

OPTIMUS addresses this gap by providing a repository-aware workflow that moves from a natural-language software issue to a context-aware, tested and verifiably correct code change, while keeping the developer in control at approval points.

1.3 Core Differentiator

Typical coding assistant	OPTIMUS
Generates code or answers questions	Works through a complete repository-level engineering task
Often context is limited to snippets/files	Builds an indexed repository context and retrieves relevant evidence
May stop after generating a suggestion	Runs tests/build/lint and uses failures as feedback
Human performs verification manually	Produces verification evidence, diff, logs and final status
Code generation is the product	Verified software change is the product

UNDERSTAND -> CONTEXT -> PLAN -> APPROVE -> IMPLEMENT -> TEST -> DIAGNOSE -> FIX -> VERIFY -> DELIVER

1.4 Objectives

- Understand repository structure, files, symbols, dependencies and test areas.
- Convert a natural-language software task into a structured implementation plan.
- Retrieve only the context required for a task rather than sending the entire repository to the model.
- Generate or modify code in a controlled temporary workspace.
- Generate or update tests where required.
- Execute tests, builds, linting and configured checks in isolation.
- Use failure evidence to propose and apply corrective iterations.
- Provide a final, human-readable verification summary and reviewable patch.
- Measure system performance with task-level success, retry and verification metrics.

1.5 Scope

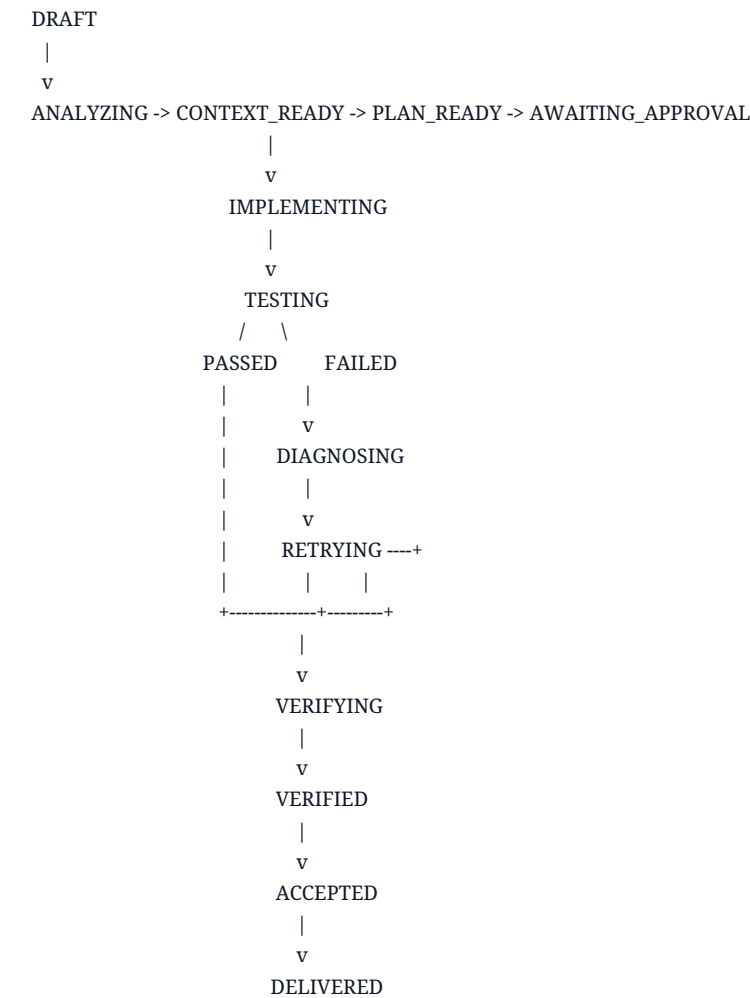
MVP	Later
JavaScript and React repositories as the initial primary ecosystem	Python, Go and other ecosystems
GitHub import and ZIP upload	Jira/GitHub Issue ingestion
Repository indexing and targeted context retrieval	Repository-wide long-term memory
Plan approval and isolated code execution	Fully automated PR merge policies
Tests/build/lint and corrective iterations	Continuous background regression monitoring
Verification and patch export	CI/CD integration and team workspaces

2. Product Workflow & State Machine

2.1 End-to-End User Journey

- 1. CONNECT REPOSITORY
- 2. INDEX REPOSITORY
- 3. CREATE TASK
- 4. UNDERSTAND TASK
- 5. BUILD CONTEXT
- 6. GENERATE PLAN
- 7. HUMAN APPROVAL
- 8. CREATE ISOLATED WORKSPACE
- 9. APPLY PATCH
- 10. RUN TESTS / BUILD / LINT
- 11. DIAGNOSE FAILURE (IF NEEDED)
- 12. CORRECTIVE ITERATION (IF APPROVED)
- 13. VERIFY
- 14. REVIEW / ACCEPT
- 15. EXPORT PATCH OR CREATE PR

2.2 Task State Machine



State rule

Implemented is not Verified. Tested is not Verified. Verified means the configured evidence supports the task completion

criteria. Accepted and Delivered are separate human/workflow states.

2.3 Human Control Gates

Gate	What the user reviews	Why
Plan Approval	Plan steps, files, assumptions, risks, test strategy	Prevents uncontrolled implementation
Patch Review	Diff, changed files, plan alignment, risks	Allows code review before verification
Corrective Iteration	Failure evidence and proposed correction	Prevents blind retry loops
Final Acceptance	Verification evidence and known limitations	Separates verified from accepted
Delivery	PR/export metadata	Separates delivery from merge/deployment

3. UI/UX Specification

3.1 Design System

Element	Decision
Theme	Dark-first developer tooling
Primary accent	Blue/indigo
Base	Near-black / deep navy
Typography	Inter/Geist for UI; JetBrains Mono for code
Tone	Technical, calm, trustworthy
Core patterns	Sidebar, dense tables, diff viewer, logs, timelines, status badges
Avoid	ChatGPT clone, robot imagery, neon overload, excessive glassmorphism

3.2 Screen Map

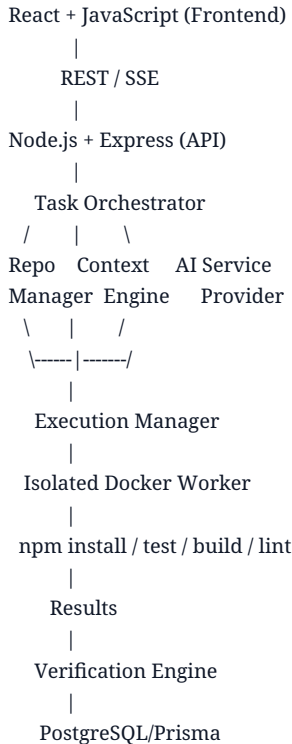
Screen	Primary responsibility
Landing	Explain product promise and repository-to-verified-change workflow
Login / Onboarding	Account and initial repository setup
Dashboard	Repository, tasks, active runs and attention overview
Repository	Overview, indexing progress and readiness state
Codebase Explorer	Search files, symbols, references, dependencies and tests
Task Workspace	Create task, view understanding, prepare context
Implementation Plan	Review and approve plan
Patch Review	Inspect side-by-side diff and plan alignment
Live Execution	High-level live task state
Execution Console	Detailed commands, logs and test progress
Failure Diagnosis	Evidence, likely cause, proposed correction
Verification	Final evidence and status
History	Tasks, runs, patches, retries and outcomes
Pull Request / Export	Deliver verified change
Notifications	Actionable events
Settings	Workspace, GitHub, AI and execution settings
Analytics / Report	Engineering metrics and task report

3.3 UX Principles

- AI never silently changes the original branch.
- Every autonomous operation exposes its current phase.
- System facts and AI hypotheses are visually distinct.
- A failure view shows evidence before recommendation.
- A verification view never claims more than the configured checks support.
- Large pages use states/tabs/drawers instead of dozens of tiny routes.

4. Technical Architecture

4.1 High-Level Architecture



4.2 Architectural Style

Use a modular monolith for the API. Each domain has a clear responsibility, but services remain in one deployable backend. Code execution is separated into a worker because it handles untrusted repository code. This keeps the project understandable for one developer while still demonstrating production-oriented isolation.

4.3 Frontend Architecture

```
frontend/
src/
  pages/
  components/
    common/
  layout/
  domain/
  features/
  hooks/
  services/
  store/
  utils/
  types/
App.jsx
main.jsx
```

React Router owns page navigation. TanStack Query owns server state. Local UI state remains local unless it must be shared across routes. Monaco Editor is used for code/diff viewing; React Flow is used for repository relationships; Recharts supports engineering analytics.

4.4 Backend Architecture

```
backend/src/  
  config.js  
  middleware.js  
  routes/  
  controllers/  
  services/  
  orchestrator.js  
  repository/  
  context/  
  ai/  
  analyzers.js  
  execution/  
  verification.js  
  utils.js  
  server.js  
backend/prisma/schema.prisma
```

4.5 Worker Architecture

```
worker/src/  
  config.js  
  workspace.js  
  executor.js  
  logger.js  
  worker.js
```

5. Repository Intelligence

5.1 Indexing Pipeline

Repository import

- > safe extraction / checkout
- > ignore generated + sensitive paths
- > file inventory
- > language detection
- > project structure detection
- > dependency metadata
- > symbol extraction
- > test detection
- > lightweight architecture map
- > persistent repository index

5.2 Context Retrieval

OPTIMUS should not send the whole repository to the LLM. The context engine starts from the task text, maps it to repository entities, retrieves candidate files and symbols, follows local relationships, adds relevant dependencies/tests, and assembles a bounded context pack.

Task text

- > keyword / path hints
- > symbol matches
- > import / call relationships
- > related tests
- > dependency context
- > ranked evidence
- > bounded prompt context

5.3 Relevance Scoring

Signal	Example weight	Notes
Direct task terms	High	File path, symbol or function names matching task
Call/import relation	High	Related to a selected symbol
Test relation	Medium	Tests that cover affected code
Directory proximity	Low	Near a matched module
Recent task history	Medium	Previously changed for similar issue
Dependency relation	Medium	Package or module used by affected code

The score is used for retrieval order, not as a claim of truth. The top-ranked context should remain reviewable in the Codebase Context UI.

5.4 Context Budget Policy

- Target a compact context pack; do not assume whole-repository prompting.
- Prefer source snippets over generated files.
- Exclude node_modules, build output, coverage, secrets and logs.
- Expand context iteratively only when the model indicates a missing dependency or when verification evidence points to another file.
- Track context size and token usage per task.

6. AI / LLM System

6.1 AI Responsibilities

AI task	Input	Output
Task understanding	Natural-language issue + repository metadata	Structured goal, constraints, likely areas
Plan generation	Task context pack	Ordered implementation plan + risks
Code reasoning	Relevant code + plan	Patch intent and code changes
Failure diagnosis	Test logs + stack trace + context	Likely cause + evidence + corrective direction
Explanation	Verified results	Concise engineering summary

6.2 Deterministic vs AI Boundary

Deterministic tooling	AI / LLM
File traversal	Issue interpretation
AST/symbol extraction	Repository reasoning
Dependency inspection	Implementation planning
Test/build/lint execution	Failure diagnosis
Exact logs and exit codes	Concise explanation
Diff capture	Code generation/refinement
Verification rule evaluation	Correction proposal
Reliability rule The LLM proposes or interprets; the application executes and verifies. The system never treats model output alone as proof of correctness.	

6.3 Provider Abstraction

```
ai/  
  provider.js    # interface/contract  
  openRouter.js  # initial implementation  
  prompts.js     # task-specific prompt builders
```

The provider abstraction isolates model-specific API details. Provider selection should be configurable through environment variables. Do not expose provider keys to the browser.

6.4 Prompt Contracts

Prompts should request structured output whenever the application consumes the result programmatically. Example plan response shape:

```

{
  "summary": "...",
  "steps": [
    {
      "title": "...",
      "files": ["..."],
      "reason": "...",
      "risk": "low|medium|high",
      "tests": ["..."]
    }
  ],
  "assumptions": ["..."],
  "openQuestions": ["..."]
}

```

Do not expose hidden chain-of-thought. Store concise evidence-backed reasons and structured outputs instead.

6.5 AI Cost Policy

Control	Rule
Context size	Bounded per task
Retries	Hard maximum, configurable
Model selection	Configurable provider/model
Caching	Repository index persisted; no blind prompt replay
Usage tracking	Store calls/tokens/cost estimate per run
Fallback	Optional provider fallback only when explicitly configured

7. Execution, Isolation & Security

7.1 Worker Lifecycle

API schedules run

- > worker allocates temp workspace
- > repository snapshot available
- > patch applied
- > dependency install (if required)
- > tests / build / lint
- > logs + result artifacts
- > worker returns result
- > workspace cleaned

7.2 Security Requirements

- Never execute uploaded repository commands in the API process.
- Use a separate Docker worker with CPU, memory and timeout limits.
- Sanitize ZIP extraction paths to prevent traversal attacks.
- Restrict network access during execution unless a task explicitly needs it.
- Store GitHub and AI credentials only server-side.
- Redact secrets and sensitive paths from logs shown to users.
- Delete temporary workspaces after a run unless retention is explicitly required.
- Use per-run identifiers so artifacts cannot leak across tasks.

7.3 Threat Model

Threat	Mitigation
Malicious ZIP	Safe extraction, path validation, size limits
Command injection	No shell interpolation from raw user text; build commands from allow-listed operations
Secret leakage	Secret scanning, .env exclusion, log redaction
Repository exfiltration	Restricted network, explicit GitHub access
Resource exhaustion	CPU/memory/time quotas
Model prompt injection in code	Treat repository text as untrusted evidence, not instructions

7.4 Execution Allow-list

For the JavaScript/TypeScript MVP, the worker should use a small allow-list of supported commands rather than executing arbitrary shell text from the model. Typical operations: install dependencies, run test, run build, run lint, inspect file/diff state. The orchestrator decides which command is allowed; the LLM cannot directly ask the worker to execute arbitrary commands.

8. Database Design & Data Model

8.1 Core Entities

Entity	Purpose
User	Account and workspace ownership
Repository	GitHub/ZIP source metadata
RepositoryBranch	Tracked branch information
Task	User-requested engineering task and state
TaskPlan	Approved plan and assumptions
TaskContext	Indexed/retrieved evidence summary
TaskFile	Files selected or changed for a task
TaskRun	Each execution attempt
Patch	Generated diff / patch metadata
TestResult	Suites, tests, exit codes, logs metadata
FailureDiagnosis	Evidence-backed failure analysis
Verification	Final verification state and evidence
Notification	Actionable task/run events
AuditHistory	State changes and event timeline

8.2 Relationship Model

```
User
|
+-- Repository
|
|   +-- RepositoryBranch
|   |
|   |   +-- Task
|   |   |
|   |   |   +-- TaskPlan
|   |   |   +-- TaskContext
|   |   |   +-- TaskFile
|   |   |   +-- TaskRun
|   |   |   |
|   |   |   |   +-- Patch
|   |   |   |   +-- TestResult
|   |   |   |   +-- FailureDiagnosis
|   |   |   |   +-- Verification
|   |   |   |   +-- AuditHistory
```

8.3 Prisma Design Rules

- Use Prisma as the single source of truth for persistence.
- Use enums for task/run/verification statuses.
- Index foreign keys and frequently queried status fields.
- Store large logs or patch artifacts outside PostgreSQL later if size requires it; keep metadata in PostgreSQL.
- Keep secrets out of database fields unless encrypted and strictly necessary.

8.4 Important Status Enums

Domain	States
Task	DRAFT, ANALYZING, CONTEXT_READY, PLAN_READY, AWAITING_APPROVAL, IMPLEMENTING, TESTING, DIAGNOSING, RETRYING, VERIFYING, VERIFIED, ACCEPTED, DELIVERED, FAILED, CANCELLED
Run	QUEUED, RUNNING, PASSED, FAILED, CANCELLED, TIMEOUT
Verification	PENDING, PASS, PARTIAL, FAIL
Patch	DRAFT, REVIEW, APPROVED, REJECTED, EXPORTED, PR_OPEN

9. REST API Specification

9.1 API Principles

- REST endpoints under /api.
- Controllers validate/authenticate; services own business logic.
- All state transitions are explicit and idempotent where possible.
- Long-running operations return a run/task identifier and stream progress separately.
- Errors use stable machine-readable codes plus user-safe messages.

9.2 Core Endpoints

Method	Endpoint	Purpose
GET	/api/me	Current user
POST	/api/auth/session	Create/refresh session
GET	/api/repositories	List repositories
POST	/api/repositories/github	Import GitHub repository
POST	/api/repositories/upload	Upload ZIP repository
GET	/api/repositories/:id	Repository details
POST	/api/repositories/:id/index	Start indexing
GET	/api/repositories/:id/tree	File tree
GET	/api/repositories/:id/search	Codebase search
POST	/api/tasks	Create task
GET	/api/tasks/:id	Task details
POST	/api/tasks/:id/analyze	Analyze task
GET	/api/tasks/:id/context	Context
POST	/api/tasks/:id/plan	Generate plan
POST	/api/tasks/:id/approve	Approve plan
POST	/api/tasks/:id/run	Start execution
GET	/api/runs/:id	Run status
GET	/api/runs/:id/logs	Run logs
POST	/api/runs/:id/diagnose	Create diagnosis
POST	/api/runs/:id/retry	Start corrective retry
GET	/api/tasks/:id/verification	Verification result
POST	/api/tasks/:id/accept	Accept verified result
POST	/api/tasks/:id/export	Export patch
POST	/api/tasks/:id/pull-request	Create GitHub PR
GET	/api/history	Task/run history
GET	/api/notifications	Notifications
GET	/api/analytics	Engineering analytics

9.3 Example API Contract - Create Task

POST /api/tasks

Content-Type: application/json

```
{
  "repositoryId": "repo_123",
  "branch": "main",
  "title": "Handle expired JWT tokens",
  "description": "Expired JWTs should return 401...",
  "type": "BUG_FIX",
  "priority": "HIGH",
  "constraints": ["Do not change valid-token behavior"]
}
```

201 Created

```
{
  "id": "task_456",
  "status": "DRAFT"
}
```

9.4 Error Contract

```
{
  "error": {
    "code": "REPOSITORY_ACCESS_DENIED",
    "message": "The repository is not accessible with the current GitHub connection.",
    "requestId": "req_123"
  }
}
```

10. Final JavaScript / React Project Structure

The implementation language is JavaScript/JSX for all application code. There is no TypeScript or TSX in the final structure.

File-count policy

Do not optimize the project for a magic number of files. Create a file only when it owns a distinct responsibility. Merge tiny abstractions when they reduce clarity.

11. Core Algorithms & Technical Logic

11.1 Repository Indexing Algorithm

```
function indexRepository(root) {  
  const files = scanSafely(root, IGNORE_PATTERNS);  
  const metadata = detectLanguagesAndProject(files);  
  const dependencies = inspectManifests(root);  
  const symbols = extractSymbols(files);  
  const tests = detectTests(files);  
  const graph = buildRelations(symbols, dependencies, tests);  
  return persistIndex({ files, metadata, dependencies, symbols, tests, graph });  
}
```

11.2 Relevant File Retrieval

1. Parse task title/description into keywords and entities
2. Search file paths and symbols
3. Expand using imports/calls/tests
4. Score candidates
5. Select top context within budget
6. Add related configuration/dependency evidence
7. Save context manifest for auditability

11.3 Plan Validation

The generated plan must be checked before approval. Validate that each step names concrete files, that the files exist, that the requested behavior is reflected, that the test strategy is present, and that the plan does not request prohibited operations.

11.4 Patch Application

The patch is applied only inside the temporary worker workspace. Before and after states are recorded. Unexpected files or directories are detected. The original repository branch is never edited directly by the worker.

11.5 Verification Rules

```
Verified =  
  planAligned  
  AND requiredTestsPassed  
  AND requiredBuildPassed  
  AND requiredLintPassed  
  AND noCriticalSecurityFinding  
  AND noUnexpectedCriticalFiles  
  AND taskNotCancelled
```

11.6 Retry Policy

Rule	Recommended default
Maximum corrective retries	2
Retry trigger	Failed verification with actionable diagnosis
Do not retry	Security-critical, ambiguous or environment failures without user approval
Preserve history	Every retry gets a new run record

Stop condition	Verified, max retries reached, cancellation or unsafe condition
----------------	---

12. Testing Strategy

12.1 Test Layers

Layer	Scope
Frontend unit	Reusable components, hooks, query states
Frontend integration	Task flow, repository flow, patch review
Backend unit	Context retrieval, plan validation, state transitions
Backend integration	Routes + services + Prisma
Worker tests	Command allow-list, workspace, cleanup
Security tests	ZIP traversal, command injection, credential leakage
End-to-end	Repository -> task -> plan -> run -> verification

12.2 Critical Test Cases

- Unauthenticated user cannot access a private repository.
- GitHub permission denial is surfaced without leaking tokens.
- ZIP files with path traversal are rejected safely.
- .env and credential files are excluded from context and logs.
- Task cannot move to implementation without plan approval.
- Worker cannot execute a command outside the supported allow-list.
- Test failure creates a diagnosable run state.
- Corrective retry preserves prior run history.
- Unexpected file changes block verification.
- Accepted state cannot exist without a verified result.

12.3 Example E2E Scenario

Given: a repository with an authentication bug
When: the user submits the issue
Then: **OPTIMUS** identifies auth.js and tests
And: generates a plan
And: waits for approval
And: applies patch in worker
And: runs tests
And: if one test fails, creates diagnosis
And: after approved retry, re-runs tests
And: verification becomes PASS only when all required checks pass

13. Evaluation & Benchmark

A personal project becomes much stronger when it is measurable. Build a small controlled task benchmark containing real JavaScript/TypeScript repository issues where expected tests or acceptance conditions are known.

Metric	Definition
Task Success Rate	Verified tasks / total tasks
Test Pass Rate	Passing required tests / executed required tests
Recovery Rate	Failed first runs later verified after correction
Average Retries	Total corrective retries / completed tasks
Mean Run Time	Median or mean wall-clock task duration
Files Changed	Average and distribution of files modified
Verification Rate	Tasks reaching verified state / started tasks
Human Rejection Rate	Plans or patches rejected by user / reviews

13.1 Benchmark Protocol

- Use fixed repositories/commits for reproducibility.
- Record baseline tests before **OPTIMUS** changes code.
- Run each task from a clean workspace.
- Do not change evaluation tests between attempts.
- Store run IDs and verification evidence.
- Report failures honestly; do not cherry-pick only successful tasks.

14.2 Minimal Observability

Event	Fields
Task event	taskId, status, timestamp
Run event	runId, stage, duration, exitCode
AI usage	provider, model, inputTokens, outputTokens, estimatedCost
Worker event	workspaceId, command, result, resource usage
Verification	checks, status, warnings

14. Local Development & Deployment

14.1 Local Environment

docker compose up --build

Recommended local services: frontend dev server, backend API, worker container and PostgreSQL. The frontend may run outside Docker during active UI development, while the worker should remain containerized for safety parity.

14.2 Environment Variables

Variable	Used by	Purpose
DATABASE_URL	Backend	Prisma/PostgreSQL connection
GITHUB_CLIENT_ID	Backend	GitHub OAuth/app configuration
GITHUB_CLIENT_SECRET	Backend	GitHub OAuth/app secret
GITHUB_CALLBACK_URL	Backend	GitHub OAuth callback
OPENROUTER_API_KEY	Backend	LLM provider key
AI_MODEL	Backend	Selected provider model
FRONTEND_URL	Backend	CORS / callback validation
WORKER_URL	Backend	Worker control endpoint
WORKSPACE_ROOT	Worker	Temporary workspace root
MAX_RUN_SECONDS	Worker	Execution timeout
MAX_CPU	Worker	CPU limit
MAX_MEMORY_MB	Worker	Memory limit

Never place server-side secrets in frontend environment variables unless the variable is intentionally public and safe. Browser code should call the backend for AI/provider operations.

14.3 Deployment Shape

```
Browser
-> Frontend host
-> Backend API
    |
    +--> PostgreSQL
    +--> GitHub API
    +--> OpenRouter
    +--> Worker
        -> Docker sandbox
```

15. Implementation Roadmap

Phase	Focus	Deliverable
0	Foundation	React/JS, Express/JS, Prisma, Docker, env, routing
1	Auth + GitHub	Login, GitHub connection, repository import
2	Repository Intelligence	Indexing, file tree, symbols, dependencies, tests
3	Task + Context	Task creation, context retrieval, codebase explorer
4	Planning	AI task understanding, implementation plan, approval
5	Execution	Worker workspace, patch, test/build/lint
6	Failure Loop	Diagnosis, corrective iteration, run history
7	Verification	Plan alignment, test aggregation, security/static checks
8	Delivery	Diff export, GitHub PR, report
9	Polish	Notifications, analytics, settings, performance, documentation

15.1 Definition of Done for MVP

- A user can sign in and connect a GitHub repository.
- The repository is indexed and searchable.
- A user can submit a real JS/TS software issue.
- **OPTIMUS** can produce a structured plan referencing concrete files.
- The user can approve the plan.
- A patch is applied only in an isolated worker.
- Configured tests/build/lint can run and return evidence.
- A failed run can produce an evidence-backed diagnosis.
- A corrective iteration can be approved and executed.
- The final result becomes Verified only when configured checks pass.
- The user can inspect the final diff and export or create a PR.
- Task history contains a complete audit trail.

16. Detailed File Responsibility Map

Path / group	Responsibility	Phase
frontend/src/pages/Landing.jsx	Public landing page	MVP
frontend/src/pages/Login.jsx	Authentication UI	MVP
frontend/src/pages/Dashboard.jsx	Workspace dashboard	MVP
frontend/src/pages/Repository.jsx	Repository overview/indexing/ready states	MVP
frontend/src/pages/CodebaseExplorer.jsx	Repository search and code intelligence	MVP
frontend/src/pages/TaskWorkspace.jsx	Task creation, context and plan views	MVP
frontend/src/pages/PatchReview.jsx	Diff review and approval	MVP
frontend/src/pages/LiveExecution.jsx	Active run overview	MVP
frontend/src/pages/Verification.jsx	Verification evidence	MVP
frontend/src/pages/History.jsx	Task/run history	MVP
frontend/src/pages/Notifications.jsx	Action center	MVP
frontend/src/pages/Settings.jsx	Account, GitHub, AI and execution settings	MVP
frontend/src/pages/NotFound.jsx	Fallback route	MVP
frontend/src/components/common.jsx	Reusable controls and feedback UI	MVP
frontend/src/components/layout.jsx	Application shell, sidebar, top nav	MVP
frontend/src/components/domain.jsx	Repository/task/context/plan/patch/execution widgets	MVP
frontend/src/features/auth.js	Auth state and API calls	MVP
frontend/src/features/repository.js	Repository APIs and hooks	MVP
frontend/src/features/task.js	Task APIs and hooks	MVP
frontend/src/features/context.js	Context APIs and hooks	MVP
frontend/src/features/execution.js	Run APIs and live updates	MVP
frontend/src/features/history.js	History and analytics queries	MVP
frontend/src/hooks.js	Shared React hooks	MVP
frontend/src/services.js	API and error handling	MVP
frontend/src/queries.js	TanStack Query configuration and keys	MVP
frontend/src/store.js	Client query cache/state wiring	MVP
frontend/src/utlis.js	Formatting and validation helpers	MVP
frontend/src/types.js	Frontend shared shapes/constants	MVP
backend/src/config.js	Environment and runtime config	MVP
backend/src/middleware.js	Auth, validation and error middleware	MVP
backend/src/routes/*	REST endpoint wiring	MVP
backend/src/controllers/*	HTTP orchestration layer	MVP
backend/src/services/*	Domain business operations	MVP
backend/src/orchestrator.js	Task lifecycle and state transitions	MVP
backend/src/repository/*	GitHub/ZIP import and workspace prep	MVP
backend/src/context/*	Indexing and retrieval	MVP
backend/src/ai/*	LLM provider and prompts	MVP
backend/src/analyzers.js	Project/dependency/security/test analysis entry point	MVP
backend/src/execution/*	Worker communication and run management	MVP
backend/src/verification.js	Verification rules and aggregation	MVP

backend/src/utils.js	Logging, errors, retry utilities	MVP
backend/src/server.js	Express bootstrap	MVP
backend/prisma/schema.prisma	Database schema	MVP
worker/src/config.js	Worker environment policy	MVP
worker/src/workspace.js	Workspace lifecycle	MVP
worker/src/executor.js	Allow-listed command execution	MVP
worker/src/logger.js	Execution log capture	MVP
worker/src/worker.js	Worker server/entry point	MVP
shared/types.js	Shared contracts/constants	MVP
infrastructure/docker-compose.yml	Local services	MVP
infrastructure/Dockerfile.api	API image	MVP
infrastructure/Dockerfile.worker	Worker image	MVP
infrastructure/.env.example	Infra variables	MVP
infrastructure/migrate.sh	Prisma migration helper	MVP
docs/ARCHITECTURE.md	Architecture reference	MVP
docs/API.md	API reference	MVP
docs/DEVELOPMENT.md	Developer setup	MVP
docs/WORKFLOW.md	Lifecycle/state reference	MVP
docs/README.md	Documentation index	MVP
README.md	Root project overview	MVP
.gitignore	Repository hygiene	MVP
package.json	Workspace scripts	MVP

17. Future Scope

- Python / Go / Java ecosystem support after the JavaScript/TypeScript core is stable.
- GitHub Issues and Jira ingestion.
- CI/CD integration and scheduled regression runs.
- Continuous repository monitoring.
- Team workspaces and shared engineering history.
- Automatic migration/refactoring workflows.
- Long-term repository memory and engineering knowledge base.
- Advanced code search and semantic dependency graphs.
- Fine-grained policy controls for enterprise environments.

18. Risks & Mitigation

Risk	Impact	Mitigation
AI generates incorrect code	High	Human approval, isolated tests, verification gate
Repository execution is unsafe	Critical	Docker sandbox, limits, timeouts, restricted network
Large context becomes expensive	High	Indexing + relevance retrieval + bounded context
Repeated retries become expensive	Medium	Retry cap + human gate + cost tracking
Model/provider unavailable	Medium	Provider abstraction + optional fallback
Task is ambiguous	Medium	Open questions + context review + explicit assumptions
Patch modifies unrelated files	High	Unexpected-change detection blocks verification
Logs reveal secrets	Critical	Redaction + protected file handling
MVP becomes too large	High	Phase boundary and JS/TS-first ecosystem

19. Final Engineering Checklist

1. React + JavaScript/JSX only for frontend application code.
2. Node.js + Express + JavaScript only for backend and worker.
3. No TypeScript or TSX.
4. PostgreSQL + Prisma is the persistence layer.
5. No unnecessary Redis/Kafka/RabbitMQ/Kubernetes in the MVP.
6. Modular monolith API with isolated worker.
7. GitHub + ZIP repository import.
8. Repository indexing and targeted context retrieval.
9. Plan approval before implementation.
10. Isolated worker execution for repository code.
11. Allow-listed execution commands.
12. Test/build/lint evidence captured.
13. Failure diagnosis separated into evidence, hypothesis and proposal.
14. Corrective iterations are bounded and persisted.
15. Verification is evidence-based.
16. Accepted and delivered are distinct from verified.
17. PR creation does not imply merge or deployment.
18. AI keys remain server-side.
19. All major state transitions are auditable.
20. Benchmark results are reproducible and honest.

Final product promise

OPTIMUS turns a real software issue into a tested, reviewable and verified software change while keeping the developer in control.

20. Why **OPTIMUS** Is a Strong Personal Project

OPTIMUS demonstrates a different skill profile from the user's existing projects. **INTELLIVORA** demonstrates an end-user AI application. **OMNIX** demonstrates a multi-agent AI platform with modular services. **OPTIMUS** demonstrates autonomous software engineering, repository intelligence, sandboxed execution, automated tests, failure feedback and verification. The project is technically ambitious but can be staged into a focused JavaScript/React MVP.

Appendix A. Recommended AI Prompt Roles

Prompt	Purpose	Output
Task Understanding	Turn issue into goal/constraints/open questions	Structured JSON
Implementation Plan	Generate file-by-file plan and test strategy	Plan JSON
Code Reasoning	Produce/refine change from approved plan	Patch intent / code
Failure Diagnosis	Interpret evidence and propose correction	Diagnosis JSON
Verification Summary	Summarize actual results only	Evidence-backed summary

Appendix B. Example Task Walkthrough

Task: "Handle expired JWT tokens correctly in the authentication flow and add regression tests."

Context found:

- backend/middleware/auth.js
- backend/services/token.js
- backend/routes/user.js
- tests/auth.test.js

Plan:

1. Separate expired-token handling.
2. Preserve valid-token behavior.
3. Add expired + invalid token tests.
4. Run targeted auth tests.
5. Run full test suite.

Execution:

- Patch applied in worker.
- 20/21 tests passed.
- One route test failed.

Diagnosis:

- Error propagation bypassed existing 401 mapping.

Correction:

- Adjust propagation, add regression assertion.

Final verification:

- 21/21 tests passed.
- Build passed.
- Lint passed.
- No unexpected files.

Outcome: VERIFIED

Appendix C. Recommended Build Order

1. Create repository and package workspaces
2. Configure React + Vite + Tailwind
3. Configure Express + Prisma + PostgreSQL
4. Add auth + GitHub connection
5. Add repository import/indexing
6. Add codebase explorer
7. Add task creation/context
8. Add AI plan generation
9. Add approval state machine
10. Add Docker worker
11. Add patch application + execution
12. Add test/build/lint aggregation
13. Add failure diagnosis + retry
14. Add verification
15. Add history + delivery
16. Add benchmark + analytics
17. Security hardening
18. Documentation + demo dataset

Appendix D. Non-Goals for MVP

- Automatic merge to main.
- Production deployment automation.
- Multi-language compiler ecosystem.
- Unbounded autonomous loops.
- Blind full-repository prompting.
- Enterprise RBAC and billing.
- Distributed microservice fleet.

Appendix E. Exact Final File Inventory

The following is the final recommended JavaScript/JSX inventory for the MVP. Unlike earlier drafts, this list uses explicit paths rather than wildcard groups. The count is calculated from the list itself and is the source of truth for the documented file count.

Exact file count: 100 unique paths.

Area	File count
Root	3
Frontend	37
Backend	41
Worker	8
Shared	2
Infrastructure	4
Docs	5
TOTAL	100

E.1 File-by-File Inventory

Exact path	Area	Phase
README.md	Root	MVP
package.json	Root	MVP
.gitignore	Root	MVP
frontend/src/pages/Landing.jsx	Frontend	MVP
frontend/src/pages/Login.jsx	Frontend	MVP
frontend/src/pages/Dashboard.jsx	Frontend	MVP
frontend/src/pages/Repository.jsx	Frontend	MVP
frontend/src/pages/CodebaseExplorer.jsx	Frontend	MVP
frontend/src/pages/TaskWorkspace.jsx	Frontend	MVP
frontend/src/pages/PatchReview.jsx	Frontend	MVP
frontend/src/pages/LiveExecution.jsx	Frontend	MVP
frontend/src/pages/Verification.jsx	Frontend	MVP
frontend/src/pages/History.jsx	Frontend	MVP
frontend/src/pages/Notifications.jsx	Frontend	MVP
frontend/src/pages/Settings.jsx	Frontend	MVP
frontend/src/pages/NotFound.jsx	Frontend	MVP
frontend/src/components/common.jsx	Frontend	MVP
frontend/src/components/layout.jsx	Frontend	MVP
frontend/src/components/domain.jsx	Frontend	MVP
frontend/src/features/auth.js	Frontend	MVP
frontend/src/features/repository.js	Frontend	MVP
frontend/src/features/task.js	Frontend	MVP
frontend/src/features/context.js	Frontend	MVP
frontend/src/features/execution.js	Frontend	MVP
frontend/src/features/history.js	Frontend	MVP
frontend/src/hooks.js	Frontend	MVP

frontend/src/services.js	Frontend	MVP
frontend/src/queries.js	Frontend	MVP
frontend/src/store.js	Frontend	MVP
frontend/src/utlis.js	Frontend	MVP
frontend/src/types.js	Frontend	MVP
frontend/src/App.jsx	Frontend	MVP
frontend/src/main.jsx	Frontend	MVP
frontend/src/assets/styles.css	Frontend	MVP
frontend/index.html	Frontend	MVP
frontend/vite.config.js	Frontend	MVP
frontend/tailwind.config.js	Frontend	MVP
frontend/postcss.config.js	Frontend	MVP
frontend/package.json	Frontend	MVP
frontend/.env.example	Frontend	MVP
backend/src/config.js	Backend	MVP
backend/src/middleware.js	Backend	MVP
backend/src/orchestrator.js	Backend	MVP
backend/src/analyzers.js	Backend	MVP
backend/src/verification.js	Backend	MVP
backend/src/utlis.js	Backend	MVP
backend/src/server.js	Backend	MVP
backend/src/routes/index.js	Backend	MVP
backend/src/routes/auth.js	Backend	MVP
backend/src/routes/repository.js	Backend	MVP
backend/src/routes/task.js	Backend	MVP
backend/src/routes/context.js	Backend	MVP
backend/src/routes/plan.js	Backend	MVP
backend/src/routes/execution.js	Backend	MVP
backend/src/routes/verification.js	Backend	MVP
backend/src/controllers/auth.js	Backend	MVP
backend/src/controllers/repository.js	Backend	MVP
backend/src/controllers/task.js	Backend	MVP
backend/src/controllers/context.js	Backend	MVP
backend/src/controllers/plan.js	Backend	MVP
backend/src/controllers/execution.js	Backend	MVP
backend/src/controllers/verification.js	Backend	MVP
backend/src/services/auth.js	Backend	MVP
backend/src/services/repository.js	Backend	MVP
backend/src/services/task.js	Backend	MVP
backend/src/services/github.js	Backend	MVP
backend/src/services/storage.js	Backend	MVP
backend/src/repository/importer.js	Backend	MVP
backend/src/repository/workspace.js	Backend	MVP
backend/src/context/engine.js	Backend	MVP
backend/src/context/indexer.js	Backend	MVP
backend/src/context/retriever.js	Backend	MVP

backend/src/ai/provider.js	Backend	MVP
backend/src/ai/openRouter.js	Backend	MVP
backend/src/ai/prompts.js	Backend	MVP
backend/src/execution/worker.js	Backend	MVP
backend/src/execution/runManager.js	Backend	MVP
backend/prisma/schema.prisma	Backend	MVP
backend/package.json	Backend	MVP
backend/jsconfig.json	Backend	MVP
backend/.env.example	Backend	MVP
worker/src/config.js	Worker	MVP
worker/src/workspace.js	Worker	MVP
worker/src/executor.js	Worker	MVP
worker/src/logger.js	Worker	MVP
worker/src/worker.js	Worker	MVP
worker/Dockerfile	Worker	MVP
worker/package.json	Worker	MVP
worker/.env.example	Worker	MVP
shared/types.js	Shared	MVP
shared/package.json	Shared	MVP
infrastructure/docker-compose.yml	Infrastructure	MVP
infrastructure/Dockerfile.api	Infrastructure	MVP
infrastructure/Dockerfile.worker	Infrastructure	MVP
infrastructure/migrate.sh	Infrastructure	MVP
docs/ARCHITECTURE.md	Docs	MVP
docs/API.md	Docs	MVP
docs/DEVELOPMENT.md	Docs	MVP
docs/WORKFLOW.md	Docs	MVP
docs/README.md	Docs	MVP

README.md
package.json
.gitignore
frontend/src/pages/Landing.jsx
frontend/src/pages/Login.jsx
frontend/src/pages/Dashboard.jsx
frontend/src/pages/Repository.jsx
frontend/src/pages/CodebaseExplorer.jsx
frontend/src/pages/TaskWorkspace.jsx
frontend/src/pages/PatchReview.jsx
frontend/src/pages/LiveExecution.jsx
frontend/src/pages/Verification.jsx
frontend/src/pages/History.jsx
frontend/src/pages/Notifications.jsx
frontend/src/pages/Settings.jsx
frontend/src/pages/NotFound.jsx
frontend/src/components/common.jsx
frontend/src/components/layout.jsx
frontend/src/components/domain.jsx
frontend/src/features/auth.js
frontend/src/features/repository.js
frontend/src/features/task.js
frontend/src/features/context.js
frontend/src/features/execution.js
frontend/src/features/history.js
frontend/src/hooks.js
frontend/src/services.js
frontend/src/queries.js
frontend/src/store.js
frontend/src/utils.js
frontend/src/types.js
frontend/src/App.jsx
frontend/src/main.jsx
frontend/src/assets/styles.css
frontend/index.html
frontend/vite.config.js
frontend/tailwind.config.js
frontend/postcss.config.js
frontend/package.json
frontend/.env.example
backend/src/config.js
backend/src/middleware.js
backend/src/orchestrator.js
backend/src/analyzers.js
backend/src/verification.js
backend/src/utils.js
backend/src/server.js
backend/src/routes/index.js
backend/src/routes/auth.js
backend/src/routes/repository.js
backend/src/routes/task.js
backend/src/routes/context.js
backend/src/routes/plan.js
backend/src/routes/execution.js
backend/src/routes/verification.js
backend/src/controllers/auth.js
backend/src/controllers/repository.js
backend/src/controllers/task.js
backend/src/controllers/context.js

```

backend/src/controllers/plan.js
backend/src/controllers/execution.js
backend/src/controllers/verification.js
backend/src/services/auth.js
backend/src/services/repository.js
backend/src/services/task.js
backend/src/services/github.js
backend/src/services/storage.js
backend/src/repository/importer.js
backend/src/repository/workspace.js
backend/src/context/engine.js
backend/src/context/indexer.js
backend/src/context/retriever.js
backend/src/ai/provider.js
backend/src/ai/openRouter.js
backend/src/ai/prompts.js
backend/src/execution/worker.js
backend/src/execution/runManager.js
backend/prisma/schema.prisma
backend/package.json
backend/jsconfig.json
backend/.env.example
worker/src/config.js
worker/src/workspace.js
worker/src/executor.js
worker/src/logger.js
worker/src/worker.js
worker/Dockerfile
worker/package.json
worker/.env.example
shared/types.js
shared/package.json
infrastructure/docker-compose.yml
infrastructure/Dockerfile.api
infrastructure/Dockerfile.worker
infrastructure/migrate.sh
docs/ARCHITECTURE.md
docs/API.md
docs/DEVELOPMENT.md
docs/WORKFLOW.md
docs/README.md

```

Appendix F. Prisma Data Model - Field-Level Specification

The Prisma schema should contain the following conceptual fields. Exact database types can be finalized during implementation, but the relationships and required semantics should remain stable.

Entity	Recommended fields	Purpose
User	id, email, name, avatarUrl, createdAt, updatedAt	Owns repositories and tasks
Repository	id, userId, provider, owner, name, url, defaultBranch, visibility, status, indexedAt, createdAt, updatedAt	Imported GitHub or ZIP source
RepositoryBranch	id, repositoryId, name, headSha, createdAt, updatedAt	Tracked branch state
Task	id, repositoryId, branch, title, description, type, priority, status, retryCount, createdAt,	Primary engineering task

	updatedAt	
TaskPlan	id, taskId, version, summary, stepsJson, assumptionsJson, risksJson, testStrategyJson, approvedAt, createdAt	Reviewable implementation plan
TaskContext	id, taskId, version, filesJson, symbolsJson, dependenciesJson, testsJson, tokenEstimate, createdAt	Context manifest used for reasoning
TaskFile	id, taskId, path, role, relevanceScore, changeType, addedLines, removedLines	Files selected or changed
TaskRun	id, taskId, runNumber, status, stage, workspaceId, startedAt, completedAt, durationMs, exitCode, errorCode	One execution attempt
Patch	id, taskRunId, baseSha, branch, diffText, summary, status, createdAt	Generated change set
TestResult	id, taskRunId, suiteName, testName, status, durationMs, exitCode, logRef	Individual test/suite result
FailureDiagnosis	id, taskRunId, confidence, evidenceJson, hypothesis, correctivePlanJson, approvedAt, createdAt	Evidence-backed diagnosis
Verification	id, taskId, taskRunId, status, checksJson, planAligned, warningsJson, limitationsJson, verifiedAt	Final evidence and status
Notification	id, userId, taskId, runId, type, title, message, readAt, createdAt	Actionable event
AuditHistory	id, taskId, runId, eventType, fromState, toState, actorType, metadataJson, createdAt	Lifecycle trace

F.1 Key Indexes

- Index Task(repositoryId, status).
- Index TaskRun(taskId, runNumber).
- Index TestResult(taskRunId, status).
- Index Notification(userId, readAt, createdAt).
- Unique Repository(provider, owner, name, userId).
- Unique RepositoryBranch(repositoryId, name).

Appendix G. API Contract Examples

Operation	Endpoint	Request	Response
Import GitHub repository	POST /api/repositories/github	{ owner, name, branch }	{ repositoryId, status: "IMPORTED" }
Start indexing	POST /api/repositories/:id/index	{ full: true }	{ runId, status: "INDEXING" }
Analyze task	POST /api/tasks/:id/analyze	{ }	{ status: "CONTEXT_READY", summary, relevantFiles[] }
Generate plan	POST /api/tasks/:id/plan	{ }	{ planId, version, steps[], risks[], testStrategy[] }
Approve plan	POST /api/tasks/:id/approve	{ planId }	{ status: "APPROVED" }
Start run	POST /api/tasks/:id/run	{ patchId }	{ runId, status: "QUEUED" }
Run diagnosis	POST /api/runs/:id/diagnose	{ }	{ diagnosisId, confidence, hypothesis, evidence[] }
Retry	POST /api/runs/:id/retry	{ diagnosisId }	{ runId, status: "QUEUED" }
Verify	GET /api/tasks/:id/verification	-	{ status, checks, planAligned, warnings, limitations }

Appendix H. Implementation Prompts by Phase

Phase	Implementation prompt
Phase 0	Create the monorepo and root scripts. Set up frontend React/Vite/JSX, backend Express/JS, Prisma/PostgreSQL, Docker Compose and environment templates. Do not add application features yet.
Phase 1	Implement authentication and GitHub connection. Store user/repository metadata in Prisma. Add repository list and connect/import UI.
Phase 2	Implement safe repository import, indexing, language detection, file tree, symbol extraction and dependency metadata for JS/TS projects.
Phase 3	Implement task creation, context retrieval, Codebase Explorer and the Task Workspace. Add persistent TaskContext records.
Phase 4	Implement AI provider abstraction and structured task-understanding/plan-generation prompts. Add plan review and approval state.
Phase 5	Implement Docker worker, workspace lifecycle, safe patch application, allow-listed commands and log streaming.
Phase 6	Implement test/build/lint aggregation, failure diagnosis, bounded corrective retry and run history.
Phase 7	Implement evidence-based verification, patch review, acceptance, export and GitHub PR creation.
Phase 8	Add analytics, notifications, engineering report, benchmark harness and security hardening.

Appendix I. Final Validation Summary

Check	Decision	Status
Application language	React + JavaScript/JSX; Node.js + JavaScript	PASS
Database	PostgreSQL + Prisma	PASS
Execution	Docker worker isolated from API	PASS
AI provider	OpenRouter through provider abstraction	PASS
Repository scope	JavaScript / TypeScript repositories; application code remains JavaScript	PASS
Human control	Plan approval + patch review + acceptance	PASS
Verification	Evidence-based state separate from Accepted/Delivered	PASS
No TypeScript/TSX	No .ts/.tsx application files in final tree	PASS
No unnecessary distributed stack	No Redis/Kafka/RabbitMQ/Kubernetes in MVP	PASS
No hidden chain-of-thought	UI stores concise rationale/evidence only	PASS